

Brain.exe - Complete Technical Knowledge Base

1. Project Overview

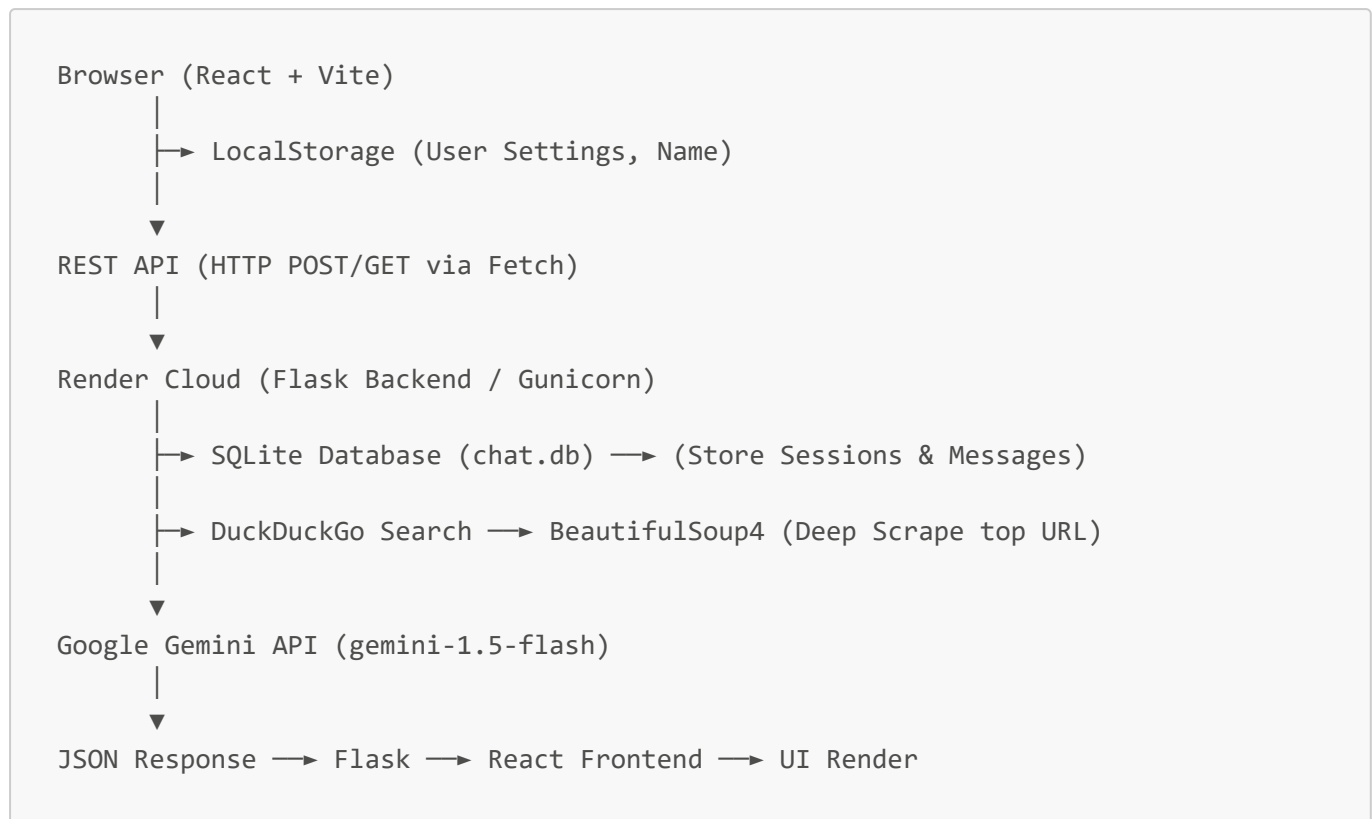
- **Project Name:** Brain.exe (Formerly Nexus AI)
- **Problem Statement:** Users need a centralized, intelligent, and highly customizable AI operating system that can perform web research, read documents, remember context, and adapt its personality to specific use cases (e.g., coding, writing, sarcastic chatting).
- **Why this project was built:** To create an "AI Operating System" that goes beyond a standard chat interface by offering mood-based persona switching, live web search integration, local document processing, and a visually stunning UI with fluid animations.
- **Target users:** Power users, developers, researchers, and individuals looking for a premium, highly responsive AI assistant.
- **Objectives:** Deliver a fast, responsive frontend, seamless backend integration, real-time web search capabilities, and a polished user experience.
- **Core features:**
 - Dynamic AI personas ("Moods")
 - Live internet search with Deep Scraping
 - Document/PDF parsing and contextual understanding
 - Persistent chat sessions and message history
 - Responsive, glassmorphism-inspired UI with Framer Motion animations

2. Complete Tech Stack

- **Programming Languages:** JavaScript (Frontend), Python 3 (Backend)
- **Frontend Framework:** React 18, Vite
- **Backend Framework:** Flask (Python)
- **Database:** SQLite
- **Authentication:** Client-side `localStorage` state simulation (mocked)
- **AI Models:** Google Gemini (`gemini-1.5-flash`)
- **LLM Provider:** Google Generative AI
- **API Keys Used:** `GEMINI_API_KEY`
- **External APIs:** DuckDuckGo Search (via `duckduckgo_search` library)
- **Libraries/Packages:**
 - Frontend: `react-router-dom`, `framer-motion`, `lucide-react`, `clsx`, `tailwind-merge`
 - Backend: `flask`, `flask-cors`, `google-generativeai`, `PyPDF2`, `beautifulsoup4`, `requests`, `gunicorn`
- **Deployment Platform:** Vercel (Frontend), Render (Backend)
- **Development Tools:** Node.js, npm, Git
- **Why this stack:** React+Vite provides a lightning-fast development experience and optimized builds. Tailwind CSS enables rapid, responsive styling. Flask is lightweight and perfect for quickly building AI microservices. SQLite is zero-configuration and great for prototyping. Gemini 1.5 Flash offers speed and massive context windows (up to 1M+ tokens), ideal for handling deep-scraped web context.

3. Overall Architecture

Architecture Diagram Flow:



Communication: The frontend sends JSON payloads via `fetch()` to Flask REST endpoints. Flask processes the business logic (DB inserts, web scraping), formats a strict prompt, calls the Gemini API, and returns a JSON response to the frontend.

4. Frontend Explanation

- **How it was built:** Bootstrapped with Vite (`npm create vite@latest`).
- **Component structure:** Modular approach (`/components/chat`, `/components/modals`).
- **Routing:** `react-router-dom` handles `/` (Landing Page) and `/chat` (Main App).
- **State management:** React `useState` and `useEffect`. `MoodContext` (Context API) manages the global AI persona state. `localStorage` is used for persisting user names and web-search toggles.
- **Styling:** Tailwind CSS with custom theme extensions (`tailwind.config.js`), utilizing glassmorphism (`backdrop-blur`, `bg-white/5`).
- **Animations:** `framer-motion` for spring-based physics (Orb pulsing, sidebar sliding, page transitions).
- **API calls:** Native `fetch` API wrapped in `async/await` inside `ChatPage.jsx`.
- **Data flow:** User types in `ChatInput` -> updates `ChatPage` state -> sends POST request to backend -> appends user message to UI -> awaits response -> appends AI message to UI.

5. Backend Explanation

- **Backend architecture:** Monolithic RESTful API.
- **Folder structure:** Single `app.py` acting as the main controller and router.
- **Routes:**
 - `GET /api/sessions`, `POST /api/sessions`, `DELETE /api/sessions/<id>`
 - `GET /api/messages/<session_id>`
 - `POST /api/chat` (Core logic)

- **POST /api/extract-text** (PDF upload)
- **Database Logic:** Native `sqlite3` driver with helper functions (`get_db()`). DB is initialized on startup.
- **Business logic:** In `/api/chat`, the server records the user message, determines if a web search is requested, scrapes data if necessary, constructs a dynamic context-aware system prompt, and queries Gemini.
- **Why this architecture:** Flask was chosen for its minimal boilerplate, allowing direct integration of Python-based AI SDKs (Gemini) and scraping tools (BeautifulSoup).

6. Frontend ↔ Backend Connection

- **Protocol:** HTTP/HTTPS.
- **API Endpoints:** Configured via `VITE_API_URL` environment variable.
- **CORS:** `flask-cors` handles cross-origin requests from the Vercel domain to the Render domain.
- **Example Request:**

```
POST /api/chat
{
  "session_id": "12345",
  "prompt": "What is the latest React version?",
  "mood": "developer",
  "web_search": true,
  "history": [{"role": "user", "content": "..."}, {"role": "model",
"content": "..."}]
}
```

- **Example Response:**

```
{
  "status": "success",
  "response": "The latest React version is 19..."
}
```

7. AI Implementation

- **LLM Used:** Google Gemini (`gemini-1.5-flash`).
- **Why this model:** High speed, massive context window (essential for web scraping), and generous free-tier limits.
- **Prompt Engineering:** The backend injects "Moods" (e.g., standard, sarcastic, developer).
- **Context Management:** If web search is enabled, the backend searches DuckDuckGo, deep-scrapes the top result using BeautifulSoup, and injects the text into the system prompt.
- **Fallback handling:** If web search fails, the backend modifies the prompt to instruct the AI to fall back to its internal knowledge (recently updated feature to prevent strict blocking).

8. API Keys

- **GEMINI_API_KEY:** The sole critical API key.

- **Purpose:** Authenticates requests to Google's Generative AI servers.
- **Storage:** Stored in Render's Environment Variables panel. Loaded via `os.environ.get("GEMINI_API_KEY")`.
- **Security:** Never exposed to the frontend. All AI calls proxy through the Flask backend.

9. Database

- **Which database:** SQLite (`chat.db`).
- **Why:** Zero configuration, serverless, stored as a local file.
- **Schema:**
 - `sessions` (id, title, mood, created_at, updated_at)
 - `messages` (id, session_id, role, content, images, created_at)
 - `projects` (id, title, description, content, status, tags, created_at)
- **Relationships:** `messages.session_id` maps to `sessions.id`.
- **Note on Render:** Render uses ephemeral filesystems for free tiers, meaning the SQLite database resets on every deployment. (Trade-off accepted for this MVP).

10. Authentication

- **Current State:** Mock authentication. The application relies on `localStorage` to simulate user state (e.g., updating the user's name to "Brain.exe" or "Aryan").
- **Future implementation:** Would require JWTs, Supabase/Firebase, or bcrypt password hashing in the backend.

11. Folder Structure

```

ChatModel/
├── src/
│   ├── components/
│   │   ├── chat/          (Sidebar, MessageList, ChatInput)
│   │   └── modals/        (ProfileModal, SettingsModal)
│   ├── context/
│   │   └── MoodContext.jsx (Global persona state)
│   ├── pages/
│   │   ├── LandingPage.jsx (Initial marketing page)
│   │   └── ChatPage.jsx    (Core application interface)
│   ├── index.css          (Tailwind entry)
│   └── main.jsx            (React entry point)
├── backend/
│   ├── app.py             (Flask server, DB logic, AI logic)
│   └── requirements.txt    (Python dependencies)
├── package.json
├── tailwind.config.js
└── vite.config.js

```

12. Complete Request Flow

1. **User clicks submit:** `ChatInput.jsx` fires `onSubmit`.

2. **Frontend State:** `ChatPage.jsx` appends user message locally and sets `isLoading = true`.
3. **API Request:** `fetch` calls `POST /api/chat`.
4. **Backend Validation:** Flask parses JSON.
5. **Database:** `app.py` inserts user message into `messages` table.
6. **Web Search (Optional):** `duckduckgo_search` finds links -> `requests` fetches HTML -> `BeautifulSoup` extracts text.
7. **AI Model:** System prompt compiled (Mood + Scraped Text) -> Sent to `gemini.generate_content()`.
8. **Backend Processing:** DB inserts AI response.
9. **Frontend:** Receives JSON response, updates state, auto-scrolls to bottom.

13. Feature-by-Feature Technical Breakdown

- **Web Search:**
 - Frontend: Toggle switch sets `isWebSearchEnabled` state.
 - Backend: Uses `DDGS().text()` and `news()`, grabs top URL, requests HTML, strips tags, truncates to 5000 chars, injects into system prompt.
- **Dynamic Moods:**
 - Frontend: `MoodContext` provides colors and IDs.
 - Backend: Dictionary `SYSTEM_PROMPTS` maps the mood ID to specific system instructions (e.g., `developer` -> "Output raw code").
- **Responsive Layout:**
 - Frontend: Tailwind classes (`flex-col sm:flex-row`). Modals were refactored to adapt their flex directions based on viewport size.

14. Important Algorithms

- **Deep Web Scraping:**
 1. Retrieve search results.
 2. Iterate through results to find the first valid `url`.
 3. Perform `requests.get()` with a 5-second timeout.
 4. Parse with `BeautifulSoup(content, 'html.parser')`.
 5. Extract all `<p>` tags, join text, and truncate `[:5000]` to prevent token overflow.

15. Prompt Engineering

- **Prompt Flow:**
 - `Base Context` (Current Date/Time) + `Mood Prompt` (Personality) + `Web Scrape Context` (If applicable) -> Final System Prompt.
- **Web Search Fallback:**
 - "If the search results aren't directly applicable, just respond naturally using your normal persona and knowledge." (Designed to prevent the AI from refusing casual chat when web search is enabled).

16. Deployment

- **Frontend (Vercel):** Connects to GitHub, builds via `npm run build`, serves static files. Environment variable `VITE_API_URL` points to backend.

- **Backend (Render):** Connects to GitHub, uses `requirements.txt` to install dependencies, runs via `gunicorn app:app --bind 0.0.0.0:$PORT`.
- **CI/CD:** Automatic redeployments on `git push origin main`.

17. Challenges

- **Module Import Errors:** Encountered `ModuleNotFoundError: No module named 'duckduckgo_search'`. Solved by strictly defining dependencies in `requirements.txt`.
- **Strict Web Search Prompts:** The AI initially refused casual conversations when Web Search was ON because it couldn't find "casual chat" on Google. Solved by relaxing the strict instruction in the system prompt.
- **Mobile Responsiveness:** Modals were squished on mobile screens. Solved by replacing hardcoded `w-48` and `flex` with responsive `w-full sm:w-48 flex-col sm:flex-row`.

18. Security

- **CORS:** Implemented to restrict frontend access.
- **Environment Variables:** API keys kept strictly on the backend server.
- **SQL Injection:** Prevented by using parameterized queries (e.g., `cursor.execute("... WHERE id = ?", (id,))`) instead of string interpolation.

19. Performance

- **Frontend Optimization:** Vite minimizes bundle size. React state minimizes re-renders by lifting state effectively (e.g., `profileUpdate` custom event listener in `MessageList`).
- **Backend Optimization:** Truncating web scrape results to 5000 characters saves massive amounts of latency in AI response generation.

20. Interview Preparation (Sample Questions)

1. **Q: Why did you choose SQLite over PostgreSQL?**
 - *Answer:* For a fast MVP, SQLite avoids external dependencies. Trade-off: Render's ephemeral disk wipes the DB, which is acceptable for this prototype phase.
2. **Q: How does the AI know current events?**
 - *Answer:* We implemented a RAG (Retrieval-Augmented Generation) pipeline. The backend queries DuckDuckGo, scrapes the HTML of the top result, and injects it into the system prompt dynamically.
3. **Q: How did you handle React state across deeply nested components?**
 - *Answer:* Used React Context for global theme/mood settings, and custom `window.dispatchEvent` for decoupled cross-component updates (like the user name change) without prop drilling.

21. Resume Explanation

- **Bullet Points:**
 - Developed a full-stack AI Assistant using React, Tailwind, and Flask, integrating Google's Gemini 1.5 API.

- Engineered a custom Web Scraping RAG pipeline using DuckDuckGo and BeautifulSoup to feed real-time internet data into the LLM context.
- Implemented dynamic persona switching and persistent chat sessions utilizing SQLite.
- **1-Minute Pitch:** "I built an AI Operating System called Brain.exe. It's a full-stack application with a React frontend and a Python backend. It features dynamic AI personalities and a real-time web-scraping engine that allows the AI to browse the internet to answer questions accurately."

22. Lessons Learned

- **System Design:** Learned the importance of cleanly separating AI logic (backend) from UI rendering (frontend).
- **Prompt Engineering:** Realized that overly strict system prompts can break fallback logic; AI instructions require nuanced phrasing.
- **DOM & Flexbox:** Deepened understanding of responsive UI layouts and managing cross-component communication effectively in React.